Deploying HashiCorp Vault on EC2 - Step-by-Step Guide

Introduction

This guide provides detailed instructions for deploying HashiCorp Vault on an AWS EC2 instance. HashiCorp Vault is a secrets management tool that secures, stores, and tightly controls access to tokens, passwords, certificates, API keys, and other sensitive data.

Prerequisites

- AWS account with permissions to create EC2 instances
- Basic knowledge of AWS services and Linux command line
- SSH key pair for connecting to EC2 instances

Step 1: Launch an EC2 Instance

1. Log in to your AWS Management Console
2. Navigate to EC2 service
3. Click "Launch Instance"
4. Choose an Amazon Machine Image (AMI)
    - Recommended: Amazon Linux 2 or Ubuntu Server 20.04 LTS
5. Select an instance type
    - Recommended minimum: t3.medium (for production environments)
6. Configure instance details:
    - VPC with internet access
    - Enable Auto-assign Public IP
7. Add storage (minimum 20GB recommended)
8. Configure security group:
    - Allow SSH (port 22) from your IP
    - Allow Vault (port 8200) from required IPs/networks
9. Review and launch the instance
10. Select your SSH key pair

Step 2: Connect to Your EC2 Instance

```
ssh -i /path/to/your-key.pem ec2-user@your-instance-public-ip
```

Step 3: Install Required Packages

Update your system and install necessary packages:

Step 4: Download and Install Vault

```
# Download the latest version of Vault and Unzip the package
# Move the executable to /usr/local/bin/
# Verify the installation
vault --version
```

```
[ec2-user@ip-10-0-0-113 ~]$ ls
[ec2-user@ip-10-0-0-113 ~]$ sudo yum update
sudo yum install -y unzip
Amazon Linux 2023 Kernel Livepatch repository                                                    133 kB/s |  15 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:01 ago on Wed Apr 23 17:21:33 2025.
Package unzip-6.0-57.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-0-113 ~]$ wget https://releases.hashicorp.com/vault/1.19.2/vault_1.19.2_linux_amd64.zip
unzip vault_1.19.2_linux_amd64.zip
--2025-04-23 17:23:35--  https://releases.hashicorp.com/vault/1.19.2/vault_1.19.2_linux_amd64.zip
Resolving releases.hashicorp.com (releases.hashicorp.com)... 18.172.64.101, 18.172.64.38, 18.172.64.46, ...
Connecting to releases.hashicorp.com (releases.hashicorp.com)|18.172.64.101|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 166215230 (159M) [application/zip]
Saving to: 'vault_1.19.2_linux_amd64.zip'

vault_1.19.2_linux_amd64.zip      100%[===========================================================>] 158.51M  87.3MB/s    in 1.8s

2025-04-23 17:23:37 (87.3 MB/s) - 'vault_1.19.2_linux_amd64.zip' saved [166215230/166215230]

Archive:  vault_1.19.2_linux_amd64.zip
  inflating: vault
  inflating: LICENSE.txt
[ec2-user@ip-10-0-0-113 ~]$ sudo mv vault /usr/local/bin/
[ec2-user@ip-10-0-0-113 ~]$ vault --version
Vault v1.19.2 (2ee4ea013b31a770a2fc421bb1e4bc74a9669185), built 2025-04-17T16:41:30Z
```

Step 5: Create a Vault Service User

```
sudo useradd --system --home /etc/vault --shell /bin/false vault
```

Step 6: Create Vault Directories

```
[ec2-user@ip-10-0-0-113 ~]$ sudo mkdir /etc/vault
sudo vi /etc/vault/vault.hcl
[ec2-user@ip-10-0-0-113 ~]$ sudo cat /etc/vault/vault.hcl
storage "file" {
  path = "/mnt/vault/data"
}

[ec2-user@ip-10-0-0-113 ~]$ sudo mkdir -p /mnt/vault
[ec2-user@ip-10-0-0-113 ~]$ sudo useradd --system --home /etc/vault vault
[ec2-user@ip-10-0-0-113 ~]$ sudo chown vault:vault /mnt/vault
[ec2-user@ip-10-0-0-113 ~]$ sudo chmod 700 /mnt/vault
[ec2-user@ip-10-0-0-113 ~]$ sudo vi /etc/vault/vault.hcl
```

Step 7: Create Vault Configuration File

```
sudo vi /etc/vault/config.hcl
```

```
[ec2-user@ip-10-0-0-113 ~]$ sudo cat /etc/vault/vault.hcl
storage "file" {
  path = "/mnt/vault/"
}

listener "tcp" {
  address     = "0.0.0.0:8200"
  tls_disable = 1
}

ui = true
disable_mlock = true
[ec2-user@ip-10-0-0-113 ~]$
```

Note: For production environments, you should configure TLS and use a more robust storage backend like AWS DynamoDB or Consul.

Step 8: Create a Systemd Service File for Vault

```
sudo vi  /etc/systemd/system/vault.service
```

```
[ec2-user@ip-10-0-0-113 ~]$ sudo vi /etc/systemd/system/vault.service
[ec2-user@ip-10-0-0-113 ~]$ sudo cat /etc/systemd/system/vault.service
[Unit]
Description=HashiCorp Vault Service- A tool for managing secrets
Documentation=https://www.vaultproject.io/
Requires=network-online.target
After=network-online.target

[Service]
User=vault
Group=vault
ExecStart=/usr/local/bin/vault server -config=/etc/vault/vault.hcl
ExecReload=/bin/kill -HUP $MAINPID
KillMode=process
Restart=on-failure
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
[ec2-user@ip-10-0-0-113 ~]$ sudo systemctl daemon-reload
sudo systemctl enable vault
sudo systemctl start vault
sudo systemctl status vault
Created symlink /etc/systemd/system/multi-user.target.wants/vault.service → /etc/systemd/system/vault.service.
● vault.service - HashiCorp Vault Service- A tool for managing secrets
     Loaded: loaded (/etc/systemd/system/vault.service; enabled; preset: disabled)
     Active: active (running) since Wed 2025-04-23 17:25:51 UTC; 107ms ago
       Docs: https://www.vaultproject.io/
   Main PID: 2667 (vault)
      Tasks: 3 (limit: 1111)
     Memory: 14.2M
```

Step 9: Start and Enable Vault Service

```
bash
sudo systemctl daemon-reload
sudo systemctl start vault
sudo systemctl enable vault
sudo systemctl status vault
```

Step 10: Initialize Vault

bash

```
export VAULT_ADDR=http://127.0.0.1:8200
```

```
[ec2-user@ip-10-0-0-113 ~]$ export VAULT_ADDR='http://10.0.0.113:8200'
export DBUS_SESSION_BUS_ADDRESS=$XDG_RUNTIME_DIR/bus
[ec2-user@ip-10-0-0-113 ~]$ vault status
Key                 Value
---                 -----
Seal Type           shamir
Initialized         false
Sealed              true
Total Shares        0
Threshold           0
Unseal Progress     0/0
Unseal Nonce        n/a
Version             1.19.2
Build Date          2025-04-17T16:41:30Z
Storage Type        file
HA Enabled          false
[ec2-user@ip-10-0-0-113 ~]$
```

```
vault operator init
```

This command will output 5 unseal keys and an initial root token. **Store these securely** - they are crucial for accessing and managing your Vault.

Step 11: Unseal Vault

When Vault starts, it is in a sealed state. You need to unseal it using the keys generated in the previous step:

bash

```
vault operator unseal <Unseal Key 1>
vault operator unseal <Unseal Key 2>
vault operator unseal <Unseal Key 3>
```

```
[ec2-user@ip-10-0-0-113 ~]$ vault operator init
Unseal Key 1: SUEnXcZvADNnun14Z3DIC2KIqKHqq7cyalf82J0ZKBrk
Unseal Key 2: 6q/mGZ3zv0a7mW7YTB/1W427zSj4bRdK1TPILTHpZ0BN
Unseal Key 3: 2POClpj9dYsX6KE3/I43zUo/C9L5bzjsv4bRe/s7iSJe
Unseal Key 4: lz/dIFYXemdg0IaxO+ysm7r3GN25bWe+iH4MysO/XABZ
Unseal Key 5: lcsJuaIIDk7YAkA1QNgsfv7LiI0yxgQ15XRkZhJ02oV8

Initial Root Token: hvs.1hXyEe4aLTFx0LISP5HR8Txq

Vault initialized with 5 key shares and a key threshold of 3. Please securely
distribute the key shares printed above. When the Vault is re-sealed,
restarted, or stopped, you must supply at least 3 of these keys to unseal it
before it can start servicing requests.

Vault does not store the generated root key. Without at least 3 keys to
reconstruct the root key, Vault will remain permanently sealed!

It is possible to generate new unseal keys, provided you have a quorum of
existing unseal keys shares. See "vault operator rekey" for more information.
[ec2-user@ip-10-0-0-113 ~]$ vault operator unseal SUEnXcZvADNnun14Z3DIC2KIqKHqq7cyalf82J0ZKBrk
Key                Value
---                -----
Seal Type          shamir
Initialized        true
Sealed             true
Total Shares       5
Threshold          3
Unseal Progress    1/3
Unseal Nonce       92b2a607-54f8-16ce-236b-e2666e033df3
Version            1.19.2
Build Date         2025-04-17T16:41:30Z
Storage Type       file
HA Enabled         false
```

Note: By default, you need 3 out of 5 keys to unseal Vault.

Step 12: Log in to Vault

```bash
vault login
```

Enter the Initial Root Token when prompted.

Step 13: Enable Audit Logging

```bash
sudo mkdir -p /var/log/vault
sudo chown vault:vault /var/log/vault
vault audit enable file file_path=/var/log/vault/audit.log
```

Step 14: Configure Environment Variables

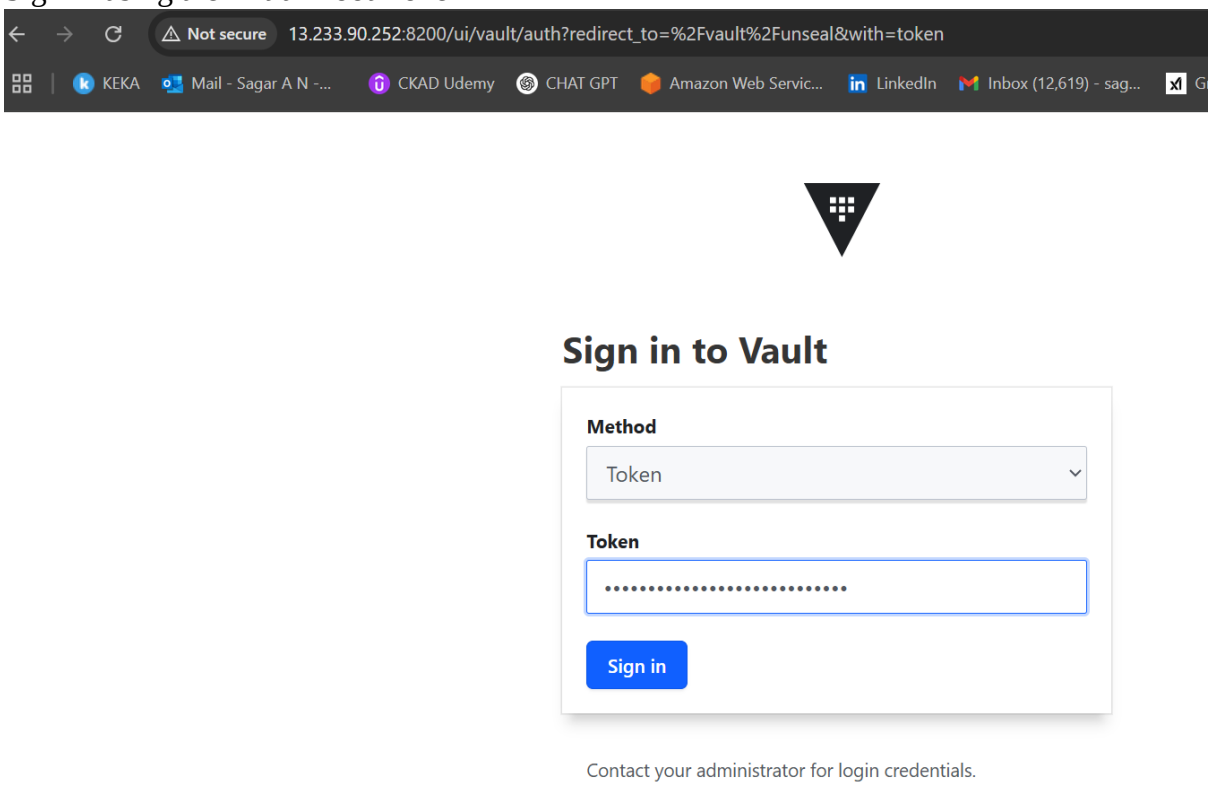
Add these lines to your `.bashrc` or `.bash_profile`:

```
export VAULT_ADDR=http://127.0.0.1:8200
```

Step 15: Access Vault Web UI

1. In your web browser, navigate to: `http://<your-instance-public-ip>:8200`

Sign in using the Initial Root Token



Enabling a Secrets Engine

Selecting a Secrets Engine

1. Navigate to "Secrets Engines" in the left navigation menu
2. Click "Enable a Secrets Engine" button
3. Available engines are organized into categories:
    - **Generic**: KV, PKI Certificates, SSH, Transit, TOTP, LDAP, Kubernetes
    - **Cloud**: AliCloud, AWS, Azure, Google Cloud, Google Cloud KMS

Configuring the KV Secrets Engine

1. Select "KV" from the engine options
2. Configure the KV engine:
    - **Path**: key-value (default path)
    - **Maximum number of versions**: Set to 0 to keep 10 versions (default)
    - **Require Check and Set**: Option to require the cas parameter on write requests
    - **Automate secret deletion**: Toggle to control whether secrets are automatically or manually deleted
3. Click "Enable engine" to complete the setup

Creating Secrets

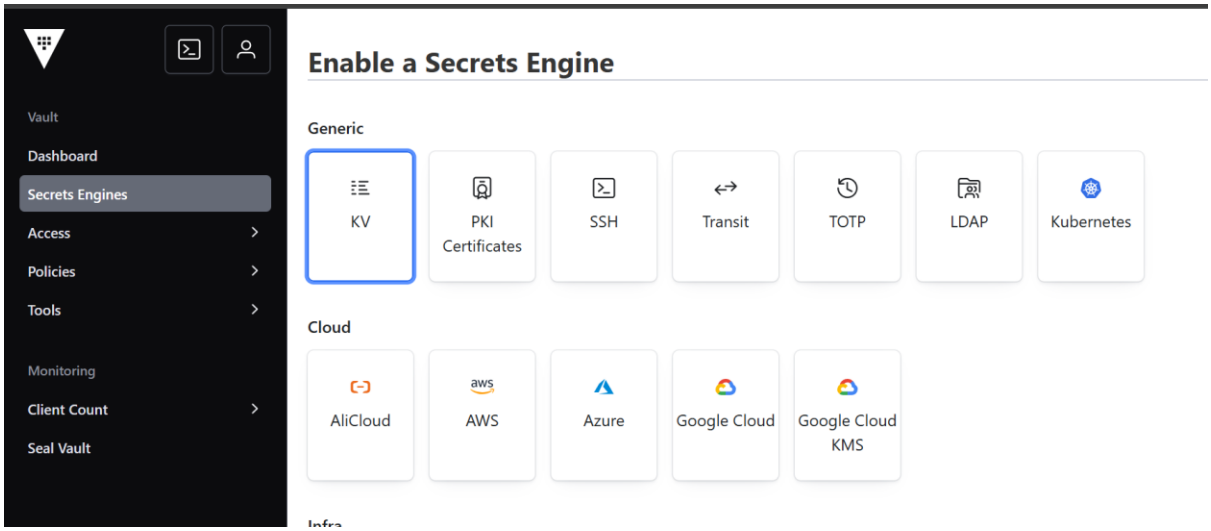
1. Navigate to the enabled KV secrets engine

2. Click "Create" to add a new secret
3. Configure the secret:
    - **Path for this secret**: dhruva-secrets-poc
    - **Secret data**: Add key-value pairs
        - Key: username
        - Value: dhruvaspace-poc
4. Click "Save" to store the secret

Viewing Secret Details

After creating a secret, you can view its details:

1. The "Overview" tab shows:
    - **Current version**: 1
    - **Secret age**: Last updated timestamp
    - **Paths**:
        - API path: /v1/key-value/data/dhruva-secrets-poc
        - CLI path: -mount="key-value" "dhruva-secrets-poc"



Vault
Dashboard
Secrets Engines
Access
Policies
Tools
Monitoring
Client Count
Seal Vault

Enable a Secrets Engine

Path

key-value

Maximum number of versions

The number of versions to keep per key. Once the number of keys exceeds the maximum number set here, the oldest version will be permanently deleted. This value applies to all keys, but a key's metadata settings can overwrite this value. When 0 is used or the value is unset, Vault will keep 10 versions.

0

Require Check and Set

If checked, all keys will require the cas parameter to be set on all write requests. A key's metadata settings can overwrite this value.

Automate secret deletion

A secret's version must be manually deleted.

Method Options

Enable engine | Back

Vault
Dashboard
Secrets Engines
Access
Policies
Tools
Monitoring
Client Count
Seal Vault

Secrets / key-value / Create

Create Secret

JSON

Path for this secret

Names with forward slashes define hierarchical path structures.

dhurva-secrets-poc

Secret data

username	dhruvaspace-poc	Add

Show secret metadata

Save | Cancel

Vault
Dashboard
Secrets Engines
Access
Policies
Tools
Monitoring
Client Count
Seal Vault

Overview | Secret | Metadata | Paths | Version History

Current version — Create new +

The current version of this secret.

1

Secret age — View metadata →

Current secret version age. Last updated on May 14 2025, 2:18:56 PM.

less than a minute

Paths

The paths to use when referring to this secret in API or CLI.

API path	/v1/key-value/data/dhurva-secrets-poc
CLI path	-mount="key-value" "dhurva-secrets-poc"

Configuring Policies

Creating Basic Policies

1. Create a policy file (policy.hcl):

   ```hcl
   path "key-value/*" {
     capabilities = ["read", "list"]
   }
   ```

2. Write the policy to Vault:

   ```bash
   vault policy write poc-policy policy.hcl
   ```

3. Create a more specific policy (poc.hcl):

   ```hcl
   # Allow reading secrets from the key-value secrets engine
   path "key-value/data/dhruva-secrets-poc" {
     capabilities = ["read"]
   }

   # Allow listing secrets under the key-value mount
   path "key-value/metadata/*" {
     capabilities = ["list"]
   }
   ```

4. Write the specific policy:

   ```bash
   vault policy write read-secret poc.hcl
   ```

5. List available policies:

   ```bash
   vault policy list
   ```

   Output shows: default, poc-policy, read-secret, root

Reading Policy Details

```bash
vault policy read read-secret
```

Setting Up Authentication Methods

Configuring AppRole Authentication

1. Enable the AppRole auth method:

   ```bash
   vault auth enable approle
   ```

2. Create an AppRole with specific parameters:

   ```bash
   vault write auth/approle/role/dhruva-role \
     token_policies="read-secret" \
     token_ttl=1h \
     token_max_ttl=4h
   ```

3. Read the role ID:

   ```bash
   vault read auth/approle/role/dhruva-role/role-id
   ```

   Output: role_id = 6fa21674-3772-8ab2-1807-ac6fea1cb3e5

4. Generate a secret ID for the role:

   ```bash
   vault write -f auth/approle/role/dhruva-role/secret-id
   ```

   Output includes:

   - secret_id: 29796b8f-e631-da0d-c681-68fe0c5aac7c
   - secret_id_accessor: 4ad9ee13-15f1-3031-1b71-227f92147813

Logging in with AppRole

```bash
vault write auth/approle/login \
  role_id="6fa21674-3772-8ab2-1807-ac6fea1cb3e5" \
  secret_id="29796b8f-e631-da0d-c681-68fe0c5aac7c"
```

```
HA Enabled       false
[ec2-user@ip-10-0-0-113 ~]$ export VAULT_TOKEN=hvs.1hXyEe4aLTFx0LISP5HR8Txq
[ec2-user@ip-10-0-0-113 ~]$ vault status
Key             Value
---             -----
Seal Type       shamir
Initialized     true
Sealed          false
Total Shares    5
Threshold       3
Version         1.19.2
Build Date      2025-04-17T16:41:30Z
Storage Type    file
Cluster Name    vault-cluster-3aaac03d
Cluster ID      1190031b-0c33-46af-a10d-17af4cf5a371
HA Enabled      false
```

Using the AppRole Token

1. Set the environment variable with the new token:

   bash

   ```bash
   export VAULT_TOKEN=hvs.CAESIPYrcImh8knS5pVXJKwORkt-xzBJiP12JT_cUJlk_KIKGh4KHGcy5STndxMmxqc0NHaWtrYkMzekFWcUhzekFWc1M
   ```

2. Verify access by retrieving the secret:

   bash

   ```bash
   vault kv get -mount=key-value dhruva-secrets-poc
   ```

```
[ec2-user@ip-10-0-0-113 ~]$ vault kv get -mount="key-value" dhurva-secrets-poc
========== Secret Path ==========
key-value/data/dhurva-secrets-poc

======= Metadata =======
Key                Value
---                -----
created_time       2025-05-14T08:48:56.493532919Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

====== Data ======
Key         Value
---         -----
username    dhruvaspace-poc
[ec2-user@ip-10-0-0-113 ~]$ vi policy.hcl
[ec2-user@ip-10-0-0-113 ~]$ cat policy.hcl
path "key-value/*" {
  capabilities = ["read", "list"]
}
```

```
[ec2-user@ip-10-0-0-113 ~]$ vault policy write poc-policy policy.hcl
Success! Uploaded policy: poc-policy
[ec2-user@ip-10-0-0-113 ~]$ vault token create -policy=poc-policy
Key                  Value
---                  -----
token                hvs.CAESID5JUefazVA0cYdgTitW9cO_QshgFqEJ9k1qMf-e_fHGGh4KHGh2cy5BOXFPU25pc2ZQUmI4TTNwd25Pb2FOYjk
token_accessor       1SSp3yn6WfHevZiccXU6YwpG
token_duration       768h
token_renewable      true
token_policies       ["default" "poc-policy"]
identity_policies    []
policies             ["default" "poc-policy"]
```

```
[ec2-user@ip-10-0-0-113 ~]$ vi poc.hcl
[ec2-user@ip-10-0-0-113 ~]$ cat poc.hcl
# Allow reading secrets from the key-value secrets engine
path "key-value/data/dhurva-secrets-poc" {
  capabilities = ["read"]
}

# Allow listing secrets under the key-value mount
path "key-value/metadata/*" {
  capabilities = ["list"]
}

[ec2-user@ip-10-0-0-113 ~]$ vault policy write read-secret poc.hcl
Success! Uploaded policy: read-secret
[ec2-user@ip-10-0-0-113 ~]$ vault policy list
default
poc-policy
read-secret
root
[ec2-user@ip-10-0-0-113 ~]$
[ec2-user@ip-10-0-0-113 ~]$ vault policy read read-secret
# Allow reading secrets from the key-value secrets engine
path "key-value/data/dhurva-secrets-poc" {
  capabilities = ["read"]
}

# Allow listing secrets under the key-value mount
path "key-value/metadata/*" {
  capabilities = ["list"]
}
```

```
[ec2-user@ip-10-0-0-113 ~]$ vault auth enable approle
Success! Enabled approle auth method at: approle/
[ec2-user@ip-10-0-0-113 ~]$ vault write auth/approle/role/dhurva-role \
    token_policies="read-secret" \
    token_ttl=1h \
    token_max_ttl=4h
Success! Data written to: auth/approle/role/dhurva-role
[ec2-user@ip-10-0-0-113 ~]$ vault read auth/approle/role/dhurva-role/role-id
Key        Value
---        -----
role_id    0fa21674-3772-8ab2-1807-ac0fea1cb3e5
[ec2-user@ip-10-0-0-113 ~]$
[ec2-user@ip-10-0-0-113 ~]$ vault write -f auth/approle/role/dhurva-role/secret-id
Key                   Value
---                   -----
secret_id             29796b8f-e631-da0d-c681-68fe0c5aac7c
secret_id_accessor    4ad9ee13-15f1-3031-1b71-227f92147813
secret_id_num_uses    0
secret_id_ttl         0s
[ec2-user@ip-10-0-0-113 ~]$ vault write auth/approle/login \
    role_id="0fa21674-3772-8ab2-1807-ac0fea1cb3e5" \
    secret_id=" 29796b8f-e631-da0d-c681-68fe0c5aac7c"
Key                     Value
---                     -----
token                   hvs.CAESIPYrcImh8KnS5pVXJKwORKt-xzBJiPi2JT_cUJlk_KIKGh4KHGh2cy5STndrMmxqc0N4aWtrYkMzekFWclhxS1M
token_accessor          AwMcNoIMGsiuSDZh2SKOqZUm
token_duration          1h
token_renewable         true
token_policies          ["default" "read-secret"]
identity_policies       []
policies                ["default" "read-secret"]
```

```
token_policies          ["default" "read-secret"]
identity_policies       []
policies                ["default" "read-secret"]
token_meta_role_name    dhurva-role
[ec2-user@ip-10-0-0-113 ~]$ export VAULT_TOKEN=hvs.CAESIPYrcImh8KnS5pVXJKwORKt-xzBJiPi2JT_cUJlk_KIKGh4KHGh2cy5STndrMmxqc0N4aWtrYkMzek
FWclhxS1M
[ec2-user@ip-10-0-0-113 ~]$ vault kv get -mount=key-value dhurva-secrets-poc
========== Secret Path ==========
key-value/data/dhurva-secrets-poc

======= Metadata =======
Key                Value
---                -----
created_time       2025-05-14T08:48:56.493532919Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1

====== Data ======
Key         Value
---         -----
username    dhruvaspace-poc
[ec2-user@ip-10-0-0-113 ~]$
```

Testing - Vault Integration with EKS for Secret Injection (EC2-hosted Vault → EKS Application)

This guide details the integration of HashiCorp Vault running on an EC2 instance with an application deployed in an EKS cluster. It enables the application to authenticate with Vault and retrieve secrets using Kubernetes ServiceAccount tokens.

Prerequisites

- Running **Vault server** on EC2 (configured and unsealed).
- An **EKS Cluster** with `kubectl` configured.
- Vault CLI access on the EC2 instance.
- Helm installed (if Vault Agent Injector is used).
- Your EKS worker nodes can access the EC2-hosted Vault.

Vault is running in an ec2 instance:

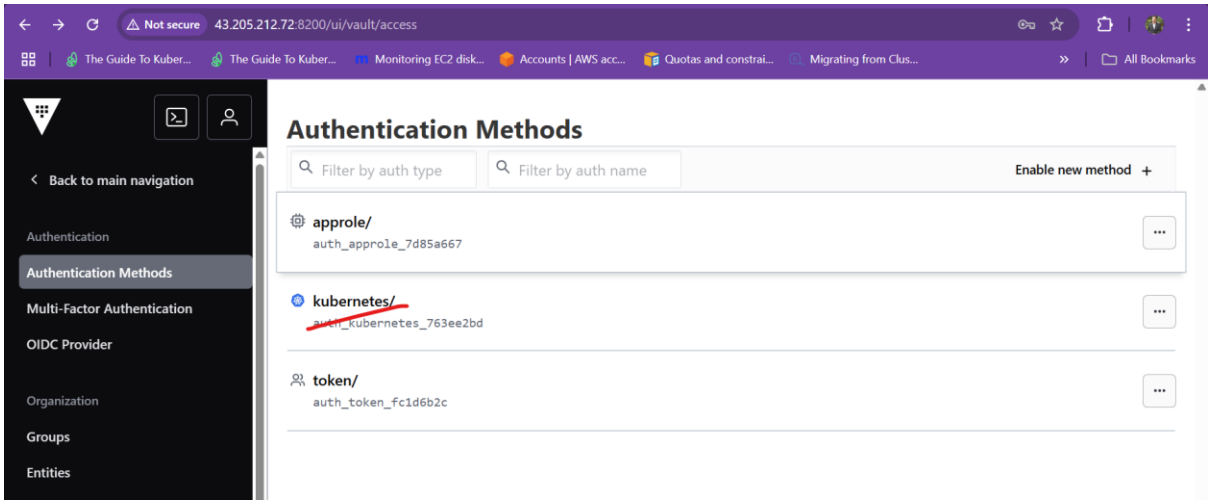
```
vault    eks
LICENSE.txt  poc.hcl  policy.hcl  read-policy.hcl  vault_1.19.2_linux_amd64.zip  vaultpolicy.hcl
[ec2-user@ip-10-0-0-113 ~]$ vault status
Key             Value
---             -----
Seal Type       shamir
Initialized     true
Sealed          false
Total Shares    5
Threshold       3
Version         1.19.2
Build Date      2025-04-17T16:41:30Z
Storage Type    file
Cluster Name    vault-cluster-3aaac03d
Cluster ID      1190031b-0c33-46af-a10d-17af4cf5a371
HA Enabled      false
[ec2-user@ip-10-0-0-113 ~]$ sudo systemctl status vault
● vault.service - HashiCorp Vault Service- A tool for managing secrets
     Loaded: loaded (/etc/systemd/system/vault.service; enabled; preset: disabled)
     Active: active (running) since Wed 2025-05-14 08:33:16 UTC; 4 days ago
       Docs: https://www.vaultproject.io/
   Main PID: 2130 (vault)
      Tasks: 7 (limit: 1111)
     Memory: 207.6M
        CPU: 18min 8.077s
     CGroup: /system.slice/vault.service
             └─2130 /usr/local/bin/vault server -config=/etc/vault/vault.hcl

May 16 06:37:53 ip-10-0-0-113.ap-south-1.compute.internal vault[2130]: 2025-05-16T06:37:53.960Z [INFO]  core: successf>
May 16 06:38:10 ip-10-0-0-113.ap-south-1.compute.internal vault[2130]: 2025-05-16T06:38:10.541Z [INFO]  core: successf>
May 16 07:31:16 ip-10-0-0-113.ap-south-1.compute.internal vault[2130]: 2025-05-16T07:31:16.850Z [INFO]  core: successf>
May 16 07:31:19 ip-10-0-0-113.ap-south-1.compute.internal vault[2130]: 2025-05-16T07:31:19.987Z [INFO]  core: successf>
May 16 07:33:42 ip-10-0-0-113.ap-south-1.compute.internal vault[2130]: 2025-05-16T07:33:42.894Z [INFO]  core: successf>
```

Make sure our eks cluster security groups can communicate with the vault ec2 instance for this add the vault port 8200 in security group of ec2

Enabled Kubernetes Auth in Vault using the command:

“vault auth enable kubernetes”



Configured Vault to Talk to EKS:

To do this we need to create one service account for vault where vault can use this service account to communicate with the pods in our eks:

Created a service account using the below yaml file:

```
[ec2-user@ip-10-5-1-237 ~]$ cat sa.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: vault-serviceaccount
  namespace: default

[ec2-user@ip-10-5-1-237 ~]$
```

we would need a service account jwt token for this we will create a new token for our vault-serviceaccount:

```
[ec2-user@ip-10-5-1-237 ~]$ cat vault-sa-token.yaml
apiVersion: v1
kind: Secret
metadata:
  name: vault-serviceaccount-token
  annotations:
    kubernetes.io/service-account.name: vault-serviceaccount
type: kubernetes.io/service-account-token

[ec2-user@ip-10-5-1-237 ~]$
```

After applying we will get a token:

```
[ec2-user@ip-10-5-1-237 ~]$ kubectl apply -f vault-sa-token.yaml
secret/vault-serviceaccount-token created
[ec2-user@ip-10-5-1-237 ~]$ kubectl get secret vault-serviceaccount-token -o jsonpath='{.data.token}' | base64 --decode
eyJhbGciOiJSUzI1NiIsImtpZCI6ImlBNFY0WlA5U1p2NzE4MlFkQXZDV2N0N2pwTEl2WWthc1hkei1BaDMxUmcifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3Nl
cnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJkZWZhdWx0Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWN
jb3VudC9zZWNyZXQubmFtZSI6InZhdWx0LXNlcnZpY2VhY2NvdW50LXRva2VuIiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY2
91bnQubmFtZSI6InZhdWx0LXNlcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQudWlkIjoiNzdlO
TlkZDYtOTlmZS00ZTc3LWE4ZGQtNmI2OGU0MTVjMjA1Iiwic3ViIjoic3lzdGVtOnNlcnZpY2VhY2NvdW50OmRlZmF1bHQ6dmF1bHQtc2VydmljZWFjY291
bnQifQ.GneHTWkI42nCNm6ZBqLYGqu-wB6oOC9RNxpaD9W5BBYyfpQfeANL3EP1T6eEufPcYh0Rlxu2mP6yjfOwzRNvnzY4vTyyUkGkma2vg6bfn9w5IPQf
sjZ40pOZ9yMQy8WZYHHxU5QZhmVXl5lKfLFHjO49vcCzu64CbhiHw66RuGIf1zKtLcIcf4NCqDs1zeh5HTlMlK0_aQe_8bc-HFZwViAUWDBr0x9vqe_50Ly
u7hEc5obTY__kX3Rf8RCzLGm856XGiWXV3gm4U7CTPFEfXZ1cETiOXvqtaQ2Wz5HBZrvOw7R-ruDKTqZKmM7Faa-Ldc503_7nDgdss91ooY7-Vw[ec2-use
r@ip-10-5-1-237 ~]$ kubectl delete deploy vault-test
```

Then we need to have a api server endpoint and cluster ca crt to find this we can run these commands:

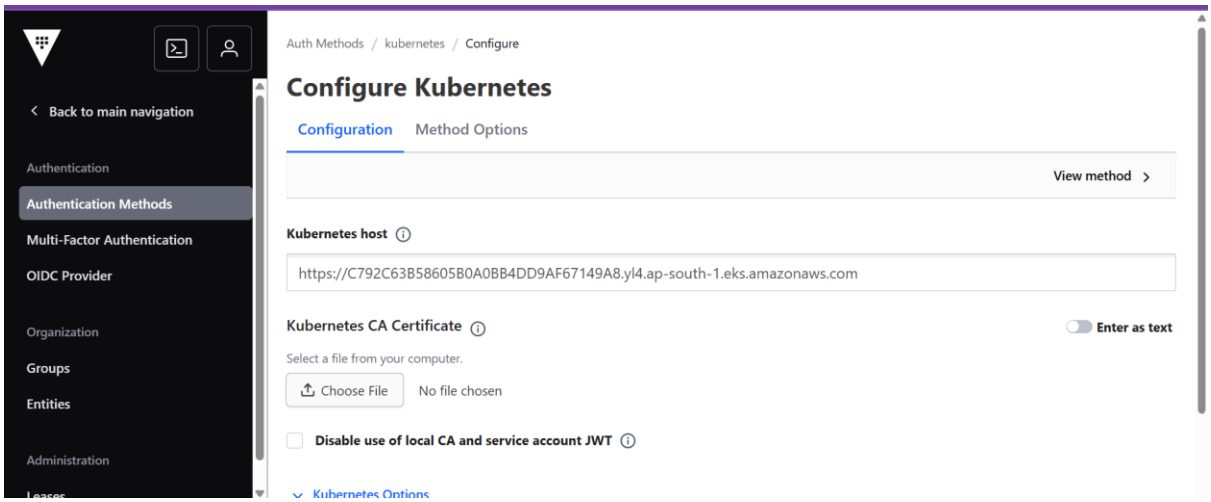
API Server endpoint:

```
[ec2-user@ip-10-5-1-237 ~]$ kubectl config view --raw --minify --flatten -o jsonpath="{.clusters[0].cluster.server}"
https://C792C63B58605B0A0BB4DD9AF67149A8.yl4.ap-south-1.eks.amazonaws.com[ec2-user@ip-10-5-1-237 ~]$ # vault-sa-token.y
```

Cluster crt:

```
[ec2-user@ip-10-5-1-237 ~]$ kubectl config view --raw -o jsonpath="{.clusters[0].cluster.certificate-authority-data}" |
 base64 --decode > /tmp/eks-ca.crt
[ec2-user@ip-10-5-1-237 ~]$ ls
ca.crt  deploy.yaml  get_helm.sh  kubectl  vault-sa-token.yaml
[ec2-user@ip-10-5-1-237 ~]$ cat /tmp/eks-ca.crt
-----BEGIN CERTIFICATE-----
MIIDBTCCAe2gAwIBAgIILQH53UuUbkUwDQYJKoZIhvcNAQELBQAwFTETMBEGA1UE
AxMKa3ViZXJuZXRlczAeFw0yNTA1MTgwODQyMjhaFw0zNTA1MTYwODQ3MjhaMBUx
EzARBgNVBAMTCmt1YmVybmV0ZXMwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEK
AoIBAQC0z5htHdY+i1AV9ANEvhFY2XWZPawR0Q6kCdggk/9AROi6BVnlEncCP3re
5h75gct75A9dtVyqLkrnt4jEasRQPmVEt/YJ5FZ6vnk0priMLnzzCF3LICFQWt4y
0kYg9PJ+oLTHV2953u6RkxfbOVjaudFDquGYCl5GVg/rouUqWGU8j4TxCD+6hQnR
SfGL7aR3YGDl5llpPBCSi+rPnxhu/hbHB/oM/SiM0Z8jPQ5McHZ+B63r6Ktg4Hqn
0bW1SCU9aRKttcqRT7DqlFxvuwAvGdsLyoU6MHw6YOisxHLaIxcZUJVprGmJsfEW
tL2rVqqjiAYNIqle7gMzhzcEJVzzAgMBAAGjWTBXMA4GA1UdDwEB/wQEAwICpDAP
BgNVHRMBAf8EBTADAQH/MB0GA1UdDgQWBBTsAcaUzKYnjgg3peXvSlcCNmkl3zAV
BgNVHREEDjAMggprdWJlcm5ldGVzMA0GCSqGSIb3DQEBCwUAA4IBAQAfzB1eyB67
L78noa2Rg6iLY5FzAP2mIkUP1unFQIuCrh2ZX/9VZoOZQGiCPNJl0pYR1fAdauRz
ClkDFre9Fd+2NI/tTCKFxv8M93d4e8+Wostp57w6d3S/DysNU95zhsd+jWnxoQ/c
DTbkGS9B/B6uANlYlufM/iF3cr8RUE1Q87ehQQfCDb5mkX8FaxQLhucDkL/9NNqW
9l2ODBlkHfSeUwfgWRJbKZmZPB6CWMcOPaNWas+ZUQJCBMuQ2pJlqHpSVqY0IkG2
t4LQPa7a9kI4FxlUd5waCmShWj0KH36fXhNrqxknX/aK1aEivRj3uE1ALx0iqLLU
RLdmmk3pA7yb
-----END CERTIFICATE-----
[ec2-user@ip-10-5-1-237 ~]$ kubectl get deploy
```

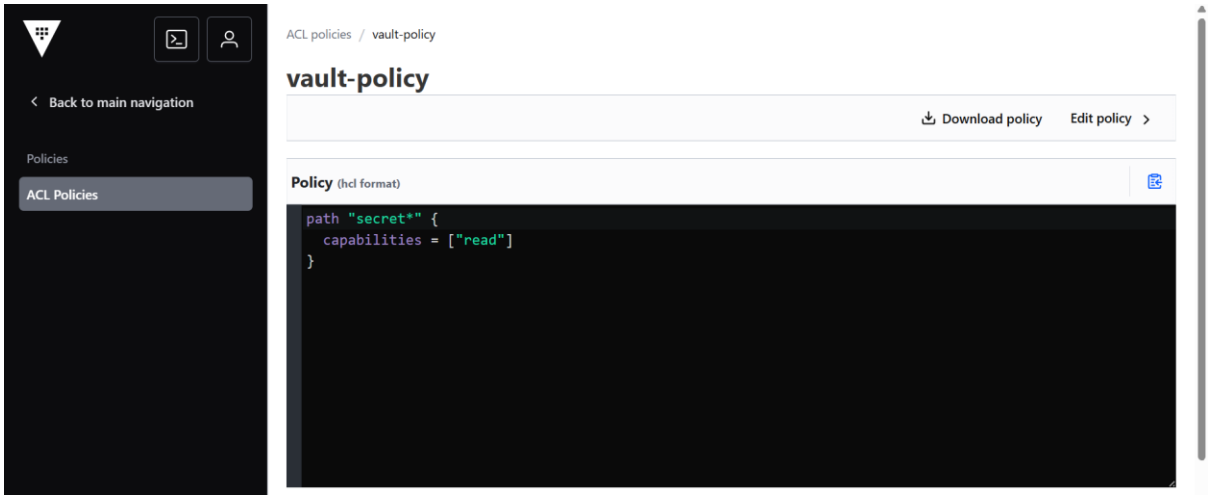
Once we get these 3 details go to the vault ui > authentication methods > kubernetes > configuration > and add these details there:



Create a vault policy:

Next, we'll create a policy that allows reading secrets. This policy will be attached to a role, which can be used to grant access to specific Kubernetes service accounts.

We can do this in ui or through cli using – "vault policy write vault-policy vault-policy.hcl"




```
[ec2-user@ip-10-0-0-113 ~]$ vault policy write vault-policy vaultpolicy.hcl
Success! Uploaded policy: vault-policy
[ec2-user@ip-10-0-0-113 ~]$ vault write auth/kubernetes/role/vault-role \
  bound_service_account_names=vault-serviceaccount \
  bound_service_account_namespaces=default \
  policies=vault-policy \
  ttl=24h
Success! Data written to: auth/kubernetes/role/vault-role
```

Create a Kubernetes vault role using this command:

Create a role(vault-role) that binds the above policy to a Kubernetes service account(vault-serviceaccount) in a specific namespace. This allows the service account to access secrets stored in Vault:

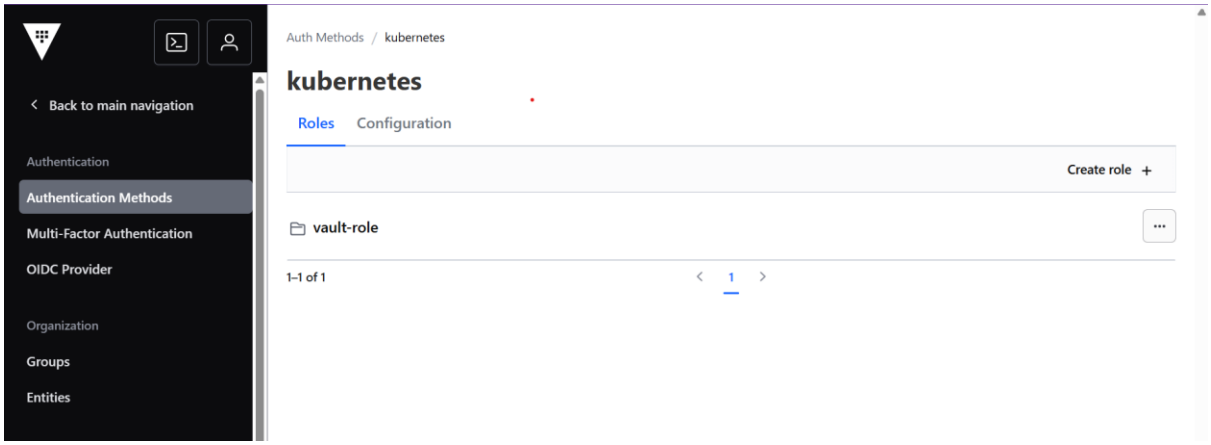
vault write auth/kubernetes/role/vault-role \

bound_service_account_names=vault-serviceaccount \

bound_service_account_namespaces=default \

policies=vault-policy \

ttl=24h



Enable Vault Injector on EKS Cluster

Install Vault Helm Chart

Use HashiCorp's Vault Helm chart to install the injector in agent mode.

helm repo add hashicorp https://helm.releases.hashicorp.com

helm repo update

```
helm install vault hashicorp/vault \
  --set "injector.enabled=true" \
  --set "server.enabled=false" \
  --set "externalVault.enabled=true" \
  --set "externalVault.address=http://<EC2_PUBLIC_IP>:8200" \
  --set "externalVault.agent=true"
```

```
[ec2-user@ip-10-5-1-237 ~]$ helm install vault hashicorp/vault \
  --set "injector.enabled=true" \
  --set "server.enabled=false" \
  --set "externalVault.enabled=true" \
  --set "externalVault.address=http://43.205.212.72:8200" \
  --set "externalVault.agent=true"
NAME: vault
LAST DEPLOYED: Sun May 18 13:34:32 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Thank you for installing HashiCorp Vault!

Now that you have deployed Vault, you should look over the docs on using
Vault with Kubernetes available here:

https://developer.hashicorp.com/vault/docs


Your release is named vault. To learn more about the release, try:

  $ helm status vault
  $ helm get manifest vault
```

Add secrets:

```
[ec2-user@ip-10-0-0-113 ~]$ vault kv put secret/login pattoken=ytbuytbytbf765rb65u56rv
== Secret Path ==
secret/data/login

======= Metadata =======
Key                Value
---                -----
created_time       2025-05-18T13:58:50.93314961Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1
[ec2-user@ip-10-0-0-113 ~]$ vault kv list secret
Keys
----
login
[ec2-user@ip-10-0-0-113 ~]$ vault kv put secret/my-first-secret username="admin" password="p@ss123"
======= Secret Path =======
secret/data/my-first-secret

======= Metadata =======
Key                Value
---                -----
created_time       2025-05-18T13:59:19.007769678Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1
[ec2-user@ip-10-0-0-113 ~]$ vault kv list secret
```

Now we will deploy one application and in this deployment file we will provide the vault annotations so that the deployment can know from where they can fetch the secrets:

This deployment manifest creates a single replica of an Nginx pod configured to securely fetch secrets from Vault. The Vault Agent injects the secrets login and my-first-secret into the pod according to the specified templates. The secrets are stored in the pod's filesystem and can be accessed by the application running in the container. The vault-serviceaccount service account, which has the necessary permissions, is used to authenticate with Vault.

```
[ec2-user@ip-10-5-1-237 ~]$ cat deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vault-test
  labels:
    app: read-vault-secret
spec:
  selector:
    matchLabels:
      app: read-vault-secret
  replicas: 1
  template:
    metadata:
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "vault-role"
        vault.hashicorp.com/agent-inject-status: "update"
        vault.hashicorp.com/vault-addr: "http://43.205.212.72:8200"
        vault.hashicorp.com/agent-inject-secret-login: "secret/login"
        vault.hashicorp.com/agent-inject-template-login: |
          {{- with secret "secret/login" -}}
          pattoken={{ .Data.data.pattoken }}
          {{- end }}
        vault.hashicorp.com/agent-inject-secret-my-first-secret: "secret/my-first-secret"
        vault.hashicorp.com/agent-inject-template-my-first-secret: |
          {{- with secret "secret/my-first-secret" -}}
          username={{ .Data.data.username }}
          password={{ .Data.data.password }}
          {{- end }}
        vault.hashicorp.com/role: "vault-role"
      labels:
        app: read-vault-secret
    spec:
      serviceAccountName: vault-serviceaccount
      containers:
      - name: nginx
        image: nginx
[ec2-user@ip-10-5-1-237 ~]$
```

These annotations are used to configure the Vault Agent to inject secrets into the pod volume.

vault.hashicorp.com/agent-inject: “true”: Enables Vault Agent injection for this pod.

vault.hashicorp.com/agent-inject-status: “update”: Ensures the status of secret injection is updated.

vault.hashicorp.com/agent-inject-secret-login: “secret/login”: Specifies that the secret stored at secret/login in Vault should be injected.

vault.hashicorp.com/agent-inject-template-login: Defines the template for the injected login secret, specifying the format in which the secret will be written.

vault.hashicorp.com/agent-inject-secret-my-first-secret: “secret/my-first-secret”: Specifies that the secret stored at secret/my-first-secret in Vault should be injected.

vault.hashicorp.com/agent-inject-template-my-first-secret: Defines the template for the injected my-first-secret, specifying the format in which the secret will be written.

vault.hashicorp.com/role: “vault-role”: Specifies the Vault role to be used for authentication.

serviceAccountName: Uses the service account vault-serviceaccount which has permissions to access Vault.

After deploying you can see the application is up and running:

```
        image: nginx
[ec2-user@ip-10-5-1-237 ~]$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
vault-agent-injector-575765576d-ggtb7   1/1     Running   0          32m
vault-test-8c7f4d7fd-8g5vs              2/2     Running   0          9s
[ec2-user@ip-10-5-1-237 ~]$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
vault-agent-injector-575765576d-ggtb7   1/1     Running   0          32m
vault-test-8c7f4d7fd-8g5vs              2/2     Running   0          13s
[ec2-user@ip-10-5-1-237 ~]$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
vault-agent-injector-575765576d-ggtb7   1/1     Running   0          32m
vault-test-8c7f4d7fd-8g5vs              2/2     Running   0          14s
[ec2-user@ip-10-5-1-237 ~]$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
vault-agent-injector-575765576d-ggtb7   1/1     Running   0          32m
vault-test-8c7f4d7fd-8g5vs              2/2     Running   0          16s
[ec2-user@ip-10-5-1-237 ~]$ kubectl logs -f vault-test-8c7f4d7fd-8g5vs -c vault-agent-init
==> Note: Vault Agent version does not match Vault server version. Vault Agent version: 1.19.0, Vault server version: 1.19.2
==> Vault Agent started! Log data will stream in below:

2025-05-18T14:57:49.229Z [INFO]  agent.sink.file: creating file sink
2025-05-18T14:57:49.229Z [INFO]  agent.sink.file: file sink configured: path=/home/vault/.vault-token mode=-rw-r----- owner=100 group=1000
==> Vault Agent configuration:

           Api Address 1: http://bufconn
                     Cgo: disabled
               Log Level: info
                 Version: Vault v1.19.0, built 2025-03-04T12:36:40Z
             Version Sha: 7eeafb6160d60ede73c1d95566b0c8ea54f3cb5a

2025-05-18T14:57:49.229Z [INFO]  agent.exec.server: starting exec server
```

Now in the vault agent init logs you can see the application pod is able to fetch the secrets from vault:

```
2025-05-18T14:57:49.229Z [INFO]  agent.sink.file: file sink configured: path=/home/vault/.vault-token mode=-rw-r----- owner=100 group=1000
==> Vault Agent configuration:

           Api Address 1: http://bufconn
                     Cgo: disabled
               Log Level: info
                 Version: Vault v1.19.0, built 2025-03-04T12:36:40Z
             Version Sha: 7eeafb6160d60ede73c1d95566b0c8ea54f3cb5a

2025-05-18T14:57:49.229Z [INFO]  agent.exec.server: starting exec server
2025-05-18T14:57:49.229Z [INFO]  agent.exec.server: no env templates or exec config, exiting
2025-05-18T14:57:49.229Z [INFO]  agent.auth.handler: starting auth handler
2025-05-18T14:57:49.229Z [INFO]  agent.auth.handler: authenticating
2025-05-18T14:57:49.229Z [INFO]  agent.sink.server: starting sink server
2025-05-18T14:57:49.229Z [INFO]  agent.template.server: starting template server
2025-05-18T14:57:49.229Z [INFO]  agent: (runner) creating new runner (dry: false, once: false)
2025-05-18T14:57:49.230Z [INFO]  agent: (runner) creating watcher
2025-05-18T14:57:49.250Z [INFO]  agent.auth.handler: authentication successful, sending token to sinks
2025-05-18T14:57:49.250Z [INFO]  agent.auth.handler: starting renewal process
2025-05-18T14:57:49.251Z [INFO]  agent.sink.file: token written: path=/home/vault/.vault-token
2025-05-18T14:57:49.251Z [INFO]  agent.sink.server: sink server stopped
2025-05-18T14:57:49.251Z [INFO]  agent: sinks finished, exiting
2025-05-18T14:57:49.251Z [INFO]  agent.template.server: template server received new token
2025-05-18T14:57:49.251Z [INFO]  agent: (runner) stopping
2025-05-18T14:57:49.251Z [INFO]  agent: (runner) creating new runner (dry: false, once: false)
2025-05-18T14:57:49.251Z [INFO]  agent: (runner) creating watcher
2025-05-18T14:57:49.251Z [INFO]  agent: (runner) starting
2025-05-18T14:57:49.253Z [INFO]  agent.auth.handler: renewed auth token
2025-05-18T14:57:49.259Z [INFO]  agent: (runner) rendered "(dynamic)" => "/vault/secrets/my-first-secret"
2025-05-18T14:57:49.260Z [INFO]  agent: (runner) rendered "(dynamic)" => "/vault/secrets/login"
2025-05-18T14:57:49.260Z [INFO]  agent: (runner) stopping
2025-05-18T14:57:49.260Z [INFO]  agent.template.server: template server stopped
2025-05-18T14:57:49.260Z [INFO]  agent.auth.handler: shutdown triggered, stopping lifetime watcher
2025-05-18T14:57:49.260Z [INFO]  agent.auth.handler: auth handler stopped
2025-05-18T14:57:49.260Z [INFO]  agent: (runner) received finish
2025-05-18T14:57:49.260Z [INFO]  agent.exec.server: exec server stopped
```

And I am able to get secrets inside our pod:

```
[ec2-user@ip-10-5-1-237 ~]$ kubectl exec -it vault-test-8c7f4d7fd-8g5vs -- sh -c "cat /vault/secrets/login && cat /vaul
t/secrets/my-first-secret"
Defaulted container "nginx" out of: nginx, vault-agent, vault-agent-init (init)
pattoken=ytbuytbytbf765rb65u56rv
username=admin
password=p@ss123
[ec2-user@ip-10-5-1-237 ~]$ ls
```

That’s it! We have successfully created secrets in Vault and utilized them within our pod.